Chapter 4

Problem reduction

Chapter 4. Problem reduction

4.1 Introduction

4.2 Node and Arc-consistency Achieving Algorithms

4.3 Path-consistency Achievement Algorithms

4.4 Post-conditions of PC Algorithms

4.5 Algorithm for Achieving k-consistency

4.6 Adaptive-consistency

4.7 Parallel/Distributed Consistency Achievement

4.1 Introduction

Problem reduction is the process of removing values from domains, and tightening constraints in a CSP, without ruling out solution tuples from a CSP.

By removing redundant values and compound labels, we *reduce* a CSP to an *equivalent problem* — a problem which has the same solution tuples as the original problem (Definitions 2-3 and 2-4) — which is hopefully easier to solve.

Problem reduction can be used

- in *preprocessing*: reducing the problem before any other techniques are applied to find solutions
- during searches — by pruning off search spaces after each label has been committed to.
- can help one to make searches backtrack free
- can help in solutions synthesis

Possible gains from problem reduction when combined with searching:

1. Reducing the search space
2. Avoiding repeatedly searching futile subtrees
3. Detecting insoluble problems

Referred to as **achieving consistency** or **problem relaxation**: remove redundant values from domains and redundant compound labels in constraints, such that consistency property holds in reduced problem.

4.2 Node and Arc-consistency Achieving Algorithms

4.2.1 Achieving NC

```
PROCEDURE NC-1(Z, D, C)
BEGIN
    FOR each x in Z
        FOR each v in D_x
            IF NOT satisfies((<x,v>), C_x)
            THEN D_x ← D_x – {v};
    return(Z, D, C);                    /* certain D_x may be updated */
END /* of NC-1 */
```

a = maximum size of domains; n = number of variables

Time complexity of NC-1 is $O(an)$.

4.2.2 A naive algorithm for achieving AC

```
PROCEDURE AC-1(Z, D, C)
BEGIN
    NC-1(Z, D, C);              /* D is possibly updated */
    Q ← {x→y | C_{x,y} ∈ C}
            /* x→y is an arc; C_{y,x} is the same object as C_{x,y} */
    REPEAT
        Changed ← False;
        FOR each x→y ∈ Q DO
            Changed ← (Revise_Domain(x→y, (Z, D, C)) OR
                Changed);
            /* side effect of Revise_Domain: D_x may be reduced */
    UNTIL NOT Changed;
    return(Z, D, C);
END /* of AC-1 */
```

Q is the list of binary-constraints to be examined

If $C_{x,y}$ is a constraint, both $x \rightarrow y$ and $y \rightarrow x$ are put into Q

```
PROCEDURE Revise_Domain(x→y, (Z, D, C)):
/* side effect: D_x in the calling procedure may be reduced*/
BEGIN
    Deleted ← False;
    FOR each a ∈ D_x DO
        IF there exists no b ∈ D_y such that satisfies((<x,a><y,b>), C_x,y)
            THEN
            BEGIN
                D_x ← D_x – {a};
                Deleted ← True;
            END
    return(Deleted)
END /* of Revise_Domain */
```

Post-condition of AC-1 is more than AC.

AC-1 achieves NC and AC (i.e. AC-1 achieves strong 2-consistency).

e edges in constraint graph; a is maximum size of domains; n variables

The worst case time complexity of AC-1 is $O(a^3ne)$
Space complexity of AC-1 is $O(e + na)$

If constraints are represented by sets of compound labels, then in the worst case one needs $O(n^2a^2)$ space to store the constraints.

4.2.3 Improved AC achievement algorithms

Idea: examine only those binary-constraints which could be affected by the removal of values

```
PROCEDURE AC-3((Z, D, C))
BEGIN
    NC-1(Z, D, C);
    Q ← {x→y | C_{x,y} ∈ C};
          /* x→y is an arc; C_{y,x} is the same object as C_{x,y} */
    WHILE (Q ≠ { }) DO
        BEGIN
            delete any element x→y from Q;
            IF Revise_Domain(x→y, (Z, D, C)) THEN
                Q ← Q ∪ {z→x | C_{z,x} ∈ C ∧ z ≠ x ∧ z ≠ y};
                /* side effect of Revise_Domain: D_x may be reduced */
        END
    return(Z, D, C);
END /* of AC-3 */
```

If *Revise_Domain((x,y))* removes any value from the domain of x, then the domain of any third variable z which is constrained by x must be examined. That is why $z \rightarrow x$ (except when $z = y$) is added to the queue Q

Lower bound of time complexity of AC-3 is $\Omega(a^2e)$.
Upper bound of time complexity of AC-3 is $O(a^3e)$.

Space complexity is $O(e + na)$.

If constraints are represented by sets of compound labels, then in the worst case one needs $O(n^2a^2)$ space to store the constraints.

4.2.4 AC-4, an optimal algorithm for achieving AC

For each value of every variable, AC4 keeps a set which contains all the variable-value pairs that it supports = *support sets* (*S*).

A table of *Counters* (*C*), which counts the number of supports that each label receives from each binary-constraint involving the subject variable.

A boolean matrix *M* (which can be referred to as the *Marker*) which marks the labels that have been rejected. An entry $M[x,v]$ is set to 1 if the label $\langle x,v \rangle$ has already been rejected, and 0 otherwise.

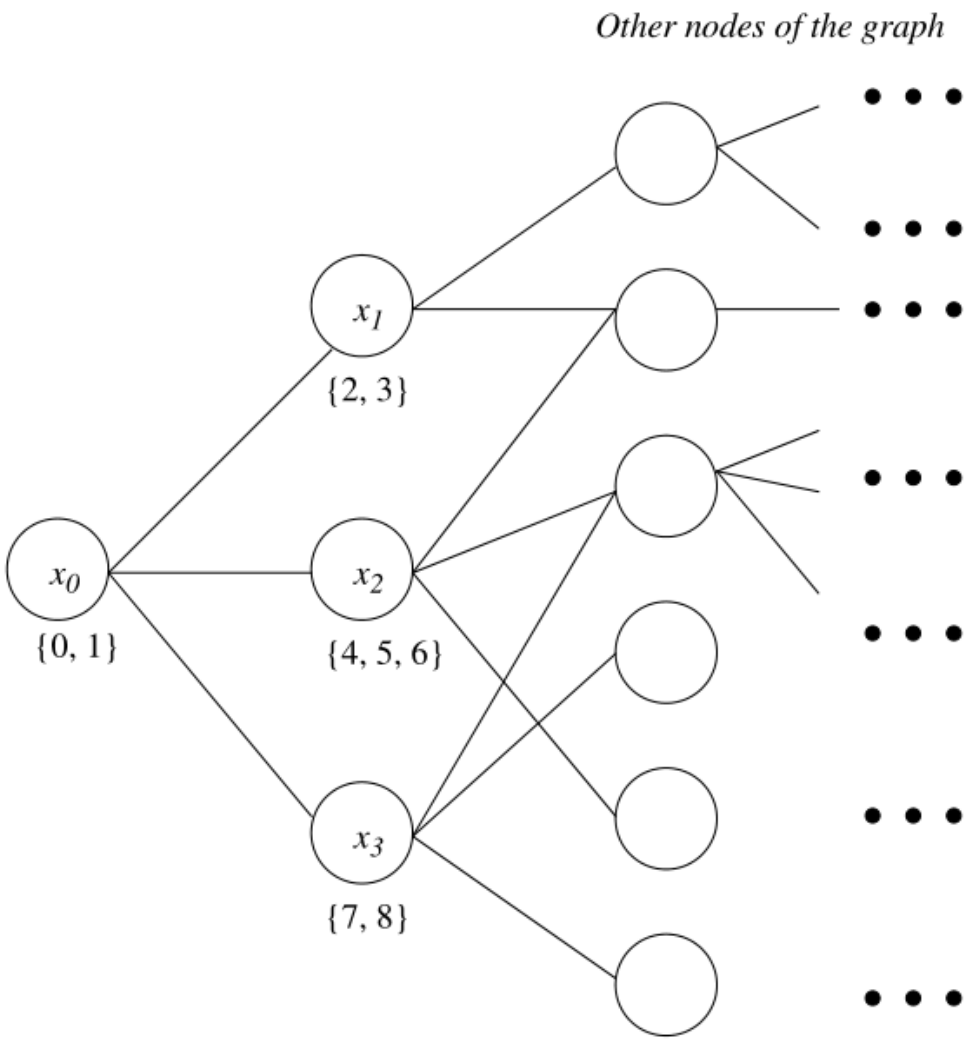


Figure 4.1 Example of a partial constraint graph. Constraints: sum of the values for the constrained variables must be even

Focus on the variable x_0. Domain of x_0 has two values, 0 and 1.

We assume only one type of constraint: the sum of values of constrained nodes must be even

$S_{\langle x0\ 0\rangle}$ = {(1,2), (2,4), (2,6), (3,8)}

$S_{\langle x0\ 1\rangle}$ = {(1,3), (2,5), (3,7)}

The support set $S_{\langle x0\ 0\rangle}$ records the fact that the label $<x_0,0>$ supports $<x_1,2>$ in variable x_1, $<x_2,4>$ and $<x_2,6>$ in variable x_2, and $<x_3,8>$ in variable x_3.

A counter is maintained for each constraint and value of each variable.

For $<x_0,0>$, the constraint $C_{x0\ x1}$ provides one support from x_1 — namely $<x_1,2>$.

Therefore, *Counter*[(0, 1), 0] = 1.

Given variables x and y such that $C_{x,y}$ is a constraint in the CSP, the *Counter*[(x,y),a] records the number of values b (in the domain of y) which are compatible with $<x,a>$.

So, we have:
Counter[(0, 1), 1] = 1
Counter[(0, 2), 0] = 2, *Counter*[(0, 2), 1] = 1
Counter[(0, 3), 0] = 1, *Counter*[(0, 3), 1] = 1.

M is initialized in the following way:
to start, every label <x,*a*> is examined using every binary constraint $C_{x,y}$ (for all *y*) in the problem. If there is no label <y,*b*> such that (<x,*a*><y,*b*>) is legal, then *a* will be deleted from the domain of x, and *M*[x,*a*] will be set to 1 (indicating that <x,*a*> has been deleted).
All rejected labels are put into a data structure called *LIST* to await further processing.
After initialization, all the labels <x,*a*> in *LIST* will be processed. Indexed by *S*, all the labels which are supported by <x,*a*> will be examined. If, according to the Counters, <x,*a*> is the only support for any label <y,*b*>, then *b* will be removed from the domain of *y*.
Any label which is rejected will be added to the *LIST*.

A label which has been processed will be deleted from *LIST*.
This process terminates when no more labels remain in *LIST*.

Back to the previous example:
If the label <x_0,1> is rejected for any reason, then M[0, 1] will be set to 1 and the label <x_0,1> will be added to the *LIST*.
When this label is processed, the support set $S_{\langle x0\ 1\rangle}$ will be looked at. Since (1, 3) is in $S_{\langle x0\ 1\rangle}$, the *Counter*[(1, 0), 3], (not the *Counter*[(0, 1), 1]), which records the support for the label <x_1,3> through the constraint $C_{x1\ x0}$, will be reduced by 1.
If a counter is reduced to 0, then the value no longer supported will be removed from its domain, and it is added to *LIST* for further processing.
In this example, if <x_0,1> is rejected, then *Counter*[(1, 0), 3] will be reduced (from 1) to 0. Therefore, 3 will be removed from the domain of x_1, M[1, 3] will be set to 1, and <x_1,3> will be put into *LIST* to await further processing.

```
PROCEDURE AC-4(Z, D, C)
BEGIN
    /* step 1: construction of M, S, Counter and LIST */
    M ← 0; S ← { };
    FOR each C_{i,j} in C DO    /* Note: C_{i,j} and C_{j,i} are the same object */
        FOR each b in D_i DO     /* examine <i,b> using variable j */
            BEGIN
                Total ← 0;
                FOR each c in D_j DO
                    IF satisfies((<i,b><j,c>), CE({i,j})) THEN
                        BEGIN
                            Total ← Total + 1;
                            S_{<j,c>} ← S_{<j,c>} + {<i,b>};
                                /* <i,b> gives support to <j,c> */
                        END;
                    IF (Total = 0) THEN          /* reject <i,b> */
                        BEGIN
                            M[i,b] ← 1;
                            D_i ← D_i – {b};
                        END
                    ELSE Counter[(i, j), b] ← Total;
                            /* support <i,b> receives from j */
            END
    LIST ← {<i,b> | M[i,b] = 1} ;
    /* LIST = set of rejected labels awaiting processing */
```

```
/* step 2: remove unsupported labels */
WHILE LIST ≠ { } DO
   BEGIN
      pick any label <j,c> from LIST; LIST ← LIST – {<j,c>};
      FOR each <i,b> in S<j,c> DO
         BEGIN
            Counter[(i, j), b] ← Counter[(i, j), b] – 1;
            IF ((Counter[(i, j), b] = 0) AND (M[i, b] = 0)) THEN
               BEGIN
                  LIST ← LIST + {<i,b>};
                  M[i,b] ← 1;
                  Di ← Di – {b};
               END;
         END
   END
return(Z, D, C);
END /* of AC-4 */
```

Step 1 of AC-4 initializes *M* (the labels that have been deleted), *S* (list of supporting labels for each label), *Counter* (number of supports for each label under each constraint) and *LIST* (the list of rejected labels awaiting processing).

The time complexity of step 1 is $O(ea^2)$.

Step 2 achieves AC by deleting labels which have no support. One rejected label <*j,c*> in *LIST* is processed at a time. Indexed by $S_{<j,c>}$, which records the list of labels that <*j,c*> supports, all the Counters of the labels which are supported by <*j,c*> are reduced by 1. If any counter is reduced to 0, the label which correspond to that counter will be rejected.

The time complexity of step 2 is ea^2.
The time complexity of AC4 is $O(ea^2)$, lower than that of AC-1 and AC-3.
The asymptotic space complexity of AC-4 is $O(a^2e)$.

4.2.5 Achieving DAC

DAC (directional arc-consistency) is a weaker property than AC.
By achieving NC and DAC, a backtrack-free search can be obtained for binary constraint problems if the constraint graphs are trees

```
PROCEDURE DAC-1(Z, D, C, <)
BEGIN
    FOR i = |Z| to 1 by –1 DO
        FOR each variable x_j where j < i AND C_{i,j} ∈ C DO
            Revise_Domain(x_j→x_i, (Z, D, C));
    return(Z, D, C);
END /* of DAC-1 */
```

DAC is defined under a total ordering (<) of the variables.

The time complexity of DAC-1 is $O(a^2e)$.
When the constraint graph is a tree of n nodes, the time complexity of DAC-1 can also be expressed as $O(a^2n)$.

A CSP P is AC if, for any given ordering of the variables <, P is DAC under both < and its reverse. Therefore, it is tempting to believe wrongly that AC could be achieved by running DAC-1 in both directions for any given <.

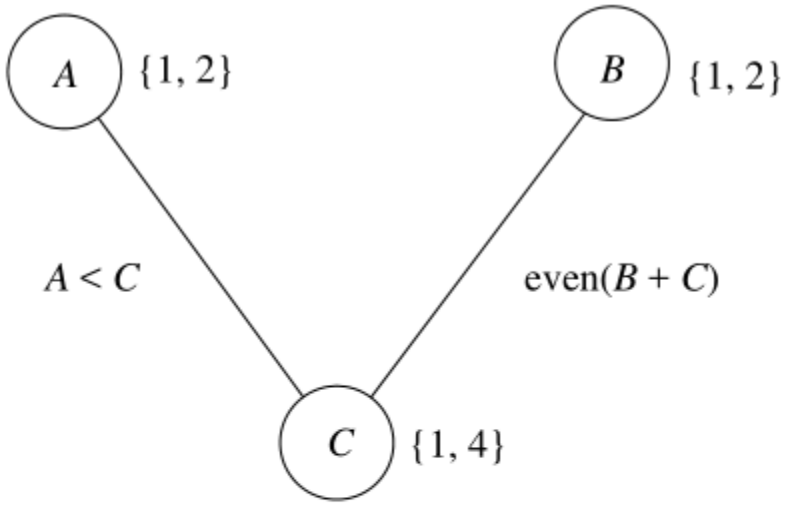


Figure 4.2 An example showing that running DAC on both directions for an arbitrary ordering does not achieve AC (After achieving DAC for both orderings (A, B, C) and (C, B, A), only <C,1> will be removed, but C has no compatible values with <B,1>, which means <B,1> should have been removed should AC be achieved.)

4.3 Path-consistency Achievement Algorithms

Algorithms which achieve path-consistency (PC) remove not only redundant values from the domains, but also redundant compound labels from the constraints.

Constraints are represented as sets of compatible 2-compound labels in these algorithms.

We introduce a relations composition mechanism which removes local inconsistency. This mechanism will be used by algorithms achieving PC.

4.3.1 Relations composition

Constraints can be represented by matrices of boolean entries.
If values in each domain have a fixed order, each entry in the matrix records the constraint on a 2-compound label.

For example, let A and B be variables in a map-colouring problem and the domain of both of them be r (for *red*) and g (for *green*) in that order. The constraint $C_{A,B}$, which specifies that $A \neq B$, can be represented by the matrix $\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$, where A takes the rows and B takes the columns, and 1 represents "legal" and 0 represents "illegal".
Given the ordering (r, g), the upper right entry (row 1, column 2) represents the fact that $\langle A,r \rangle$ and $\langle B,g \rangle$ are compatible with each other.

Both the domain and the unary constraint of a variable X are represented in the form of a binary constraint $C_{X,X}$.

The domain is represented by a matrix with 1's on no entries other than the upper left to lower right diagonal.

The relations composition mechanism ensures that a compound label (<*A*,*a*><C,*c*>) is allowed only if for all variables *B* there exists a value b such that satisfies(<*B*,*b*> C_B), satisfies((<*A*,*a*><*B*,*b*>), $C_{A,B}$), satisfies((<*B*,*b*><C,*c*>), $C_{B,C}$) all hold.

We use $C_{X,Y,r,s}$ to denote the *r*-th row, *s*-th column of $C_{X,Y}$.
We use "*" to denote a composition operation.

The composition mechanism is defined as follows:

$$\text{if } C_{X,Z} = C_{X,Y} * C_{Y,Z},$$
$$\text{then } C_{X,Z,r,s} = (C_{X,Y,r,1} \wedge C_{Y,Z,1,s}) \vee (C_{X,Y,r,2} \wedge C_{Y,Z,2,s}) \vee \ldots \vee (C_{X,Y,r,t} \wedge C_{Y,Z,t,s})$$

where *t* is the cardinality of D_Y, and "∧", "∨" are logical *AND* and *OR*.

For example, if $C_{X,Y} = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 1 \end{bmatrix}$ and $C_{Y,Z} = \begin{bmatrix} 0 & 1 & 0 \\ 1 & 1 & 1 \\ 0 & 1 & 0 \end{bmatrix}$, then $C_{Y,Z,1,1} = (1 \wedge 0) \vee (0 \wedge 1) \vee (0 \wedge 1) = 0$. The matrix $C_{X,Z}$ as composed by $C_{X,Y}$ and $C_{Y,Z}$ is $\begin{bmatrix} 0 & 1 & 0 \\ 1 & 1 & 1 \\ 0 & 1 & 0 \end{bmatrix}$.

The operation is like ordinary matrix multiplication except that number multiplication and addition are replaced by logical AND and OR.

The value that A and C can take simultaneously is constrained by the constraint $C_{A,C}$, plus the conjunction of all $C_{A,X} * C_{X,X} * C_{X,C}$ for all $X \in Z$, i.e.:

$$C_{A,C} = C_{A,C} \wedge C_{A,X_1} * C_{X_1,X_1} * C_{X_1,C} \wedge C_{A,X_2} * C_{X_2,X_2} * C_{X_2,C} \wedge \ldots \wedge C_{A,X_n} * C_{X_n,X_n} * C_{X_n,C}$$

where n is the number of variables in the problem. We call a matrix M composed by $C_{A,B}$, $C_{B,B}$ and $C_{B,C}$ if $M = C_{A,B} * C_{B,B} * C_{B,C}$.

4.3.2 PC-1, a naive PC Algorithm

Path-consistency achievement involves removing

- redundant values from domains
- redundant 2-compound labels from the binary constraints

```
PROCEDURE PC-1(Z, D, C)
/* (Z, D, C) is a binary CSP */
BEGIN
    n ← |Z|; Y^n ← C;
    REPEAT
        Y^0 ← Y^n;
        FOR k ← 1 TO n DO
            FOR i ← 1 TO n DO
                FOR j ← 1 TO n DO
                    Y_{i,j}^k ← Y_{i,j}^{k-1} ∧ Y_{i,k}^{k-1} * Y_{k,k}^{k-1} * Y_{k,j}^{k-1};
    UNTIL Y^n = Y^0;
    C ← Y^n;
    return(Z, D, C);
END /* of PC-1 */
```

The basic idea is as follows: for every variable k, pick every constraint $C_{i,j}$ from the current set of constraints Y^k and attempt to reduce it by means of relations composition using $C_{i,k}$, $C_{k,k}$ and $C_{k,j}$. After this is done for all the variables, the set of constraints is examined to see if any constraint in it has been changed. The whole process is repeated as long as some constraints have been changed.

As a maximum there could be a^2n^2 iterations in the REPEAT loop of PC-1.
Relations composition is called n^3 times in each iteration.
a^3 binary operations are required in each relations composition call.

The time complexity of PC-1 is therefore $O(a^5n^5)$.
PC-1 needs n^2a^2 space to store the constraints.
The overall space complexity of PC-1 is then $O(n^3a^2)$.

4.3.3 PC-2, an improvement over PC-1

PC-2 assumes an ordering (<) among the variables:

```
PROCEDURE PC-2(Z, D, C, <)
BEGIN
    Q ← {(i, k, j) | i, j, k ∈ Z ∧ i ≤ j ∧ (i ≠ k ≠ j)};
    WHILE (Q ≠ { }) DO
        BEGIN
            pick and delete a path (i, k, j) from Q;
            IF Revise_Constraint((i, k, j), (Z, D, C))
            THEN Q ← Q ∪ RELATED_PATHS((i, k, j), |Z|, <);
            /* side effect of Revise_Constraint: C_{i,j} may be reduced */
        END
    return(Z, D, C);
END /* of PC-2 */
```

Q is a queue of paths awaiting processing

Revise_Constraint((*i, k, j*), (*Z, D, C*)) restricts $C_{i,j}$ using $C_{i,k}$ and $C_{k,j}$:

```
PROCEDURE Revise_Constraint((i, k, j), (Z, D, C))
/* attempt to reduce Ci,j */
BEGIN
    Temp = C_{i,j} ∧ C_{i,k} * C_{k,k} * C_{k,j};
    IF (Temp = C_{i,j}) THEN return (False)
    ELSE BEGIN
                C_{i,j} ← Temp; return (True) ;
          END
END /* of Revise_Constraint */
```

RELATED_PATHS((i, k, j), n, <) in PC-2 returns the set of paths to be re-examined when $C_{i,j}$ is reduced. If $i < j$, all the paths which contain (i, j) or (j, i) are relevant, with the exception of (i, j, j) and (i, i, j) because $C_{i,j}$ will not be further restricted by these paths as a result of itself being reduced. If $i = j$, the path restricted by Revise_Constraint was (i,k,i), all the paths with i in it need to be re-examined, with the exception of (i, i, i) and (k, i, k). This is because $C_{i,i}$ will not be further restricted. $C_{k,k}$ will not be further restricted because it was the variable k which has caused $C_{i,i}$ to be reduced.

```
PROCEDURE RELATED_PATHS((i, k, j), n, <)
BEGIN
    IF (i < j) THEN
        S ← {(i, j, m) | (i ≤ m ≤ n) ∧ (m ≠ j)} ∪
                {(m, i, j) | (1 ≤ m ≤ j) ∧ (m ≠ i)} ∪
                {(j, i, m) | j < m ≤ n} ∪
                {(m, j, i) | 1 ≤ m < i};
    ELSE          /* it is the case that i = j */
        S ← {(p, i, m) | (1 ≤ p ≤ m) ∧ (1 ≤ m ≤ n)} – {(i, i, i), (k, i, k)};
    return (S);
END /* of  Related_Paths */
```

The lower bound of time complexity of PC-2 is $\Omega(a^3n^3)$ (if the problem is already PC).

The number of new entries to Q is bounded by $O(a^2n^3)$. Each call of Revise_Constraint goes through each path of length 2, its worst case time complexity is $O(a^3)$, and overall worst case time complexity of PC-2 is $O(a^5n^3)$.

The space complexity of PC-2 is $O(n^3+n^2a^2)$.

4.3.4 Further improvement of PC achievement algorithms

The efficiency of the PC-2 algorithm can be improved in the same way as AC-3 is improved to AC-4. The improved algorithm is called PC-4.

Similar to AC-4, PC-4 maintains four data structures:

1. Sets of supports, S — one for each 2-compound label;
2. Counters — one for each variable for each 2-compound label in which it is involved;
3. Markers, M — one for each 2-compound label;
4. $LIST$ — the set of 2-compound labels to be processed

As before, unary constraints are represented by $C_{i,i}$ for uniformity.

A support set $S_{<i,b><j,c>}$ is maintained for every 2-compound label ($<i,b><j,c>$). Elements of $S_{<i,b><j,c>}$ are labels $<k,d>$ (for some variable k) which is supported by the compound label ($<i,b><j,c>$). Whenever ($<i,b><j,c>$) is removed from constraint $C_{i,j}$, the compound label ($<i,b><k,d>$)

loses its support from <*j,c*>, and the compound label (<*j,c*><*k,d*>) loses its support from <*i,b*>. If any compound label loses all its supports from a variable, then it has to be rejected.

Counter[(*i,a,j,b*), *k*] is a counter for the 2-compound label (<*i,a*><*j,b*>) with regard to variable *k*. It counts the number of labels that variable *k* may take in order to support (<*i,a*><*j,b*>) (i.e. possible labels <*k,d*> which satisfies C_{ik} and C_{kj}).

A table of Markers *M* is maintained to mark those 2-compound labels which have been rejected but not yet processed. *M*[*i,b,j,c*] is set to 1 if (<*i,b*><*j,c*>) has been rejected but such a constraint has not been propagated to other compound labels; it is set to 0 otherwise.

Finally, *LIST* is the set of 2-compound labels which have been rejected but not yet processed.

```
PROCEDURE PC-4(Z, D, C)
BEGIN
    /* step 1: initialization */
    M ← 0; Counter ← 0; n = |Z| ;
    FOR all S DO S ← { };
    FOR each C_{i,j} ∈ C
        FOR k = 1 TO n DO
            FOR each b ∈ D_i DO
                FOR each c ∈ D_j such that satisfies((<i,b><j,c>), C_{i,j})
                    holds DO
                    BEGIN
                        Total ← 0;
                        FOR each d ∈ D_k DO
                            IF (satisfies((<i,b><k,d>), C_{i,k}) & satis-
                                fies((<k,d><j,c>), C_{k,j}))
                                THEN BEGIN
                                        Total ← Total + 1;
                                        S_{<i,b><k,d>} ← S_{<i,b><k,d>} + {<j,c>};
                                        S_{<j,c><k,d>} ← S_{<j,c><k,d>} + {<i,b>};
```

```
                        END
                IF Total = 0 THEN
                    BEGIN
                        M[i,b,j,c] ← 1; M[j,c,i,b] ← 1;
                        C_{i,j} ← C_{i,j} – (<i,b><j,c>);
                    END;
                ELSE BEGIN
                        Counter[(i,b,j,c),k] ← Total;
                        Counter[(j,c,i,b),k] ← Total;
                    END;
            END
    Give the variables an arbitrary order <;
    LIST ← {(<i,b><j,c>) | (M[i,b,j,c] = M[j,c,i,b] = 1) ∧ (i < j)}
    /* LIST = the set of 2-compound labels to be processed */

    /* step 2: propagation */
    WHILE LIST ≠ { } DO
        BEGIN
            pick and delete an element (<k,d><l,e>) from LIST;
            FOR each <j,c> in S_{<k,d><l,e>} DO
                PC-4-Update((<k,d><l,e>), <j,c>));
            FOR each <j,c> in S_{<l,e><k,d>} DO
                PC-4-Update((<l,e><k,d>), <j,c>));
        END /* of WHILE */
    return(Z, D, C);
END /* of PC-4 */
```

```
PROCEDURE PC-4-Update((<i,b><j,c>), <k,d>)
/* This procedure updates Counter, S, M, LIST, which are all assumed
    to be global variables, with respect to the rejection of the com-
    pound label (<i,b><j,c>). It focuses on the edge C_ik. */
BEGIN
    Counter[(i,b,k,d), j] ← Counter[(i,b,k,d), j] – 1;
    Counter[(k,d,i,b), j] ← Counter[(k,d,i,b), j] – 1;
    S_<i,b><j,c> ← S_<i,b><j,c> – {<k,d>};
    S_<k,d><j,c> ← S_<k,d><j,c> – {<i,b>};
    IF (Counter[(i,b,k,d), j] = 0) AND (M[i,b,k,d] = 0) THEN
        BEGIN
            M[i,b,k,d] ← 1; M[k,d,i,b] ← 1;
            LIST ← LIST + {(<i,b><k,d>)};
            C_i,k ← C_i,k – {(<i,b><k,d>)};
        END
END /* of PC-4-Update */
```

Step 1 is the initialization stage. The time complexity of step 1 is $O(n^3a^3)$, Step 2 achieves PC. There are $O(n^3a^2)$ counters, each having at most a values, the maximum number of times that the counters can be reduced is $O(n^3a^3)$. This is the worst case time complexity of step 2.
Combining the results, the worst case time complexity of the whole algorithm is $O(n^3a^3)$.

The space complexity of PC-4 is dominated by the number of support sets:

$$na \times \sum_{(i,j) \in N \times N} |D_i| \times |D_j|$$

which is $\leq n^3a^3$. So the space complexity of PC-4 is $O(n^3a^3)$.

4.3.5 GAC4: problem reduction for general CSPs

Mohr & Masini [1988] propose the algorithm GAC4, a modification of AC-4, for removing redundant compound labels from **general constraints**.

When a label or 2-compound label *CL* is removed, GAC4 removes from all the constraints those tuples which have *CL* as their projections.

For example, if (<*x*,*a*><*y*,*b*>) is removed from $C_{x,y}$, then for all variables *z* and values *c* (<*x*,*a*><*y*,*b*><*z*,*c*>) is removed from $C_{x,y,z}$ whenever it exists.

Besides, GAC4 removes all the labels and 2-compound labels which are not subsumed by any element of the higher order constraints in which the subject variables are involved.

For example, if (<*x*,*a*><*y*,*b*><*z*,*c*>) is removed from the constraint $C_{x,y,z}$, and there exists no value *d* such that (<*x*,*d*><*y*,*b*><*z*,*c*>) is in $C_{x,y,z}$, then (<*y*,*b*><*z*,*c*>) is removed from $C_{b,c}$.

4.3.6 Achieving DPC

Directional Path-consistency (DPC) is weaker than PC.

Achieving NC and DPC can help achieving backtrack-free search in certain problems

```
PROCEDURE DPC-1(Z, D, C, <)
/* for simplicity, assuming that for all i, j, i < j ⇔ z_i < z_j */
BEGIN
    E ← {x→y | C_{x,y} ∈ C ∧ x < y };
    FOR k = |Z| to 1 by −1 DO
        BEGIN
            /* Step (a): remove redundant values from domains */
            FOR i = 1 to k DO
                IF ((z_i→z_k) ∈ E) THEN C_{i,i} ← C_{i,i} ∧ C_{i,k} * C_{k,k} * C_{k,i};
            /* Step (b): remove redundant 2-compound labels from con-
                straints */
            FOR i = 1 to k DO
                FOR j = i to k DO
```

```
            IF ((z_i→z_k) ∈ E AND (z_j→z_k) ∈ E) THEN
                BEGIN
                    C_i,j ← C_i,j ∧ C_i,k * C_k,k * C_k,j;
                    E ← E + {z_i→z_j};
                END
        END /* of outer for loop */
    return(Z, D, C);
END /* of DPC-1 */
```

The FOR loop in step (b) will go through $O(n^2)$ combinations of i and j. Each relations composition in step (b) will examine each of the 3-compound labels. So there will be $O(a^3)$ 3-compound labels to examine.
Step (a) goes through n variables only, and it does no more relations composition than step (b).
The complexity of the outer FOR loop is dominated by step (b)
The time complexity of DPC-1 is $O(n^3a^3)$.

The length of the list E is bounded by n^2. The space complexity of DPC-1

is dominated by the binary constraints, which is $O(n^2a^2)$, the space required to represent all the constraints in CSP in the worst case.

The DPC-1 procedure has a lower time and space complexity than PC-1 and PC-2, and same time but lower space complexity than PC-4.

DPC-1 is unable to remove as many redundant values and redundant 2-compound labels as PC achievement algorithms.

The choice of achieving PC or DPC is domain dependent.

In general, more redundant values and compound labels can be removed through constraint propagation in more tightly constrained problems.

So in general, the tighter a problem, the more worthwhile it is to achieve PC.

4.4 Post-conditions of PC Algorithms

The post-condition of the PC-1, PC-2 and PC-4 procedures are in fact stronger than PC. The post-condition of DPC-1 is also stronger than DPC.

Given any problem *P1* = (*Z*, *D*, *C*), the above PC achievement procedures return an equivalent problem *P2* = (*Z*, *D'*, *C'*) which is NC, AC and PC.

1. *P2* is AC
2. *P2* should be PC
3. All solutions in *P2* satisfy *P1*
4. All solutions in *P1* satisfy *P2*

Running PC-1 before a search starts (which is referred to as *preprocessing* in searching) may improve search efficiency.
By achieving NC, AC and PC, PC-1 removes local inconsistencies which would otherwise be repeatedly discovered in backtracking search.

4.5 Algorithm for Achieving *k*-consistency

Algorithms for achieving *k*-consistency could be valuable for some applications. Cooper [1989] proposes the KS-1 algorithm.

```
PROCEDURE KS-1( Z, D, C, k )        /* achieving k-consistency */
BEGIN
    /* Step 1: initialization */
    Set ← { }; M ← 0;

    FOR i = 1 to k DO
        FOR each i-tuple X^i = (x_1,...,x_i) of variables x_1 < ... < x_i DO
            FOR each i-tuple V^i = (v_1,v_2,...,v_i) of values v_1, ..., v_i DO
                BEGIN
                    FOR each y ∈ (Z – {x_1,x_2,...,x_i}) DO
                        Counter[X^i,V^i,y] ← |D_y|;
                    IF NOT satisfies((<x_1,v_1>...<x_i,v_i>), C_{x_1...x_i}) THEN
                        BEGIN Set ← Set + {(X^i,V^i,i)}; M[X^i , V^i] ← 1; END
                END;
    /* Set stores a set of redundant i-compound-labels, indexed by i */
```

```
    /* Step 2: constraint propagation*/
    WHILE Set ≠ { } DO
        BEGIN
            Remove any (X^i,V^i,i) from Set, where X^i = (x_1,...,x_i) and V^i =
                (v_1,...,v_i);
            KS_Upward_Propagate(X^i, V^i, i, k);
            KS_Downward_Propagate(X^i, V^i, i, k)
        END
    return(Z, D, C);
END /* of KS-1 */
```

```
PROCEDURE KS_Upward_Propagate(X^i, V^i ,i, k)
/* X^i = (x_1,...,x_i) and V^i = (v_1,...,v_i), X^i and V^i together represents a
    redundant compound label which has been rejected. KS_Upward_Propagate examines i + 1 compound labels. Z, D, C, Set
    and M are treated as global variables. */
BEGIN
    IF (i < k) THEN
        FOR each <x',v'> such that x' ∉ {x_1,x_2,...,x_i} DO
            BEGIN
                X^{i+1} ← (x_1,...,x_i,x'); V^{i+1} ← (v_1,...,v_i,v');
                IF (M[X^{i+1}, V^{i+1}] = 0) THEN
                    BEGIN
                        Set ← Set + {(X^{i+1},V^{i+1},i+1)}; M[X^{i+1}, V^{i+1}] = 1;
                        C_{x_1...x_i x'} ← C_{x_1...x_i x'} −{(<x_1,v_1>...<x_i,v_i><x',v'>)};
                    END
            END
END  /* of KS_Upward_Propagate */
```

```
PROCEDURE KS_Downward_Propagate(X^i, V^i, i, k)
/* X^i = (x_1,...,x_i) and V^i = (v_1,...,v_i), X^i and V^i together represents a
    redundant compound label which has been rejected. KS_Down-
    ward_Propagate examines i - 1 compound labels. Z, D, C, Coun-
    ter, Set and M are treated as global variables. */
BEGIN
    IF (i > 1) THEN
        FOR j = 1 to i DO
            BEGIN
                X^(i-1) ← X^i with x_j removed; V^(i-1) ← V^i with v_j removed;
                Counter[X^(i-1),V^(i-1),x_j] ← Counter[X^(i-1),V^(i-1),x_j] – 1;
                IF (Counter[X^(i-1),V^(i-1),x_j] = 0) AND (M[X^(i-1), V^(i-1)] = 0) THEN
                    BEGIN
                        Set ← Set + {(X^(i-1),V^(i-1), i – 1)}; M[X^(i-1), V^(i-1)] = 1;
                        C_{x_{i-1}} ← C_{x_{i-1}} – {(<x_1,v_1>...<x_{i-1},v_{i-1}>)};
                    END
            END
END  /* of KS_Downward_Propagate */
```

If a compound label $cl = (\langle x_1,v_1\rangle \ldots \langle x_i,v_i\rangle)$ is identified to be redundant and therefore rejected, all compound labels in which *cl* is a projection will be rejected. Besides, all projections of *cl* will be examined.

Similar data structures to those used in PC-4 are maintained in KS-1.

- X^i and V^i are taken as *i*-tuples of variables and *i*-tuples of values respectively.
- *Set* is a set of (X^i,V^i,i).
- Counters count the number of supports that are given by each variable *x* to each compound label that does not include *x*.
- All *Counter*s are initialized to the domain sizes of the supporting variables. The algorithm KS-1 makes $Counter[(x_1,\ldots,x_i), (v_1,\ldots,v_i), x_j]$ equal to the number of v_j's such that satisfies$((\langle x_1,v_1\rangle\ldots\langle x_i,v_i\rangle\langle x_j,v_j\rangle), C_{x1\ \ldots xixj})$ holds.

In KS-1, consistency is achieved by restricting general constraints.
When KS-1 terminates, some k-constraints in C may have been tightened.

Both the time and space complexity of KS-1 are $O\left(\sum_{i=1}^{k}\left(C_n^i a^{i+1}\right)\right)$.

To achieve k-consistency for a higher k requires more computation.
It is only worth doing if it results in removing enough redundant compound labels to sufficiently increase search efficiency. This tends to be the case in problems which are tightly constrained.

4.6 Adaptive-consistency

For general CSPs, one can ensure that a search is backtrack-free by achieving a property called *adaptive-consistency*.

Extended definition of constraint graphs to general CSPs. Every CSP is associated with a *primal graph,* which is defined below.

Definition 4-1:

The **constraint graph** of a general CSP (Z, D, C) is an undirected graph in which each node represents a variable in Z, and for every pair of distinct nodes which corresponding variables are involved in any k-constraint in C there is an edge between them. The constraint graph of a general CSP P is also called a **primal graph** of P. We continue to use $G(P)$ to denote the constraint graph of the CSP P:

$\forall$ graph((V, E)): (V, E) = G((Z, D, C)) $\equiv$ (($V = Z$) $\wedge$ E = {(x,y) | x, $y \in Z \wedge$ ($\exists$ C_S $\in$ C: x, $y \in S$)}) ■

Definition 4-2:
The Parents of a variable x under an ordering is the set of all nodes which precede x according to the ordering and are adjacent to x in the primal graph:

$\forall$ csp((Z, D, C)):
($\forall <$: total_ordering(Z, $<$): (V, E) = primal_graph((Z, D, C)):
($\forall\ x \in V$: parents(x, (V, E), $<$) $\equiv \{y \mid y < x \wedge (x,y) \in E\}$)) ■

Definition 4-3:
A CSP P is adaptive-consistent under a total-ordering of its variables if for all variables x, there exists a constraint C_S on the parents of x (S), and every compound label in C_S satisfies all the relevant constraints on S.

$\forall$ csp((Z, D, C)): ($\forall <$: total_ordering(Z, $<$):
(adaptive-consistent((Z, D, C), $<$) $\equiv$
($\forall\ x \in Z$: ($S = \{y \mid y < x \wedge \exists\ C_{S'} \in C$: $x, y \in S'\} \Rightarrow$
$\exists\ C_S \in C$: $\forall\ cl \in C_S$: satisfies(cl, CE(S, (Z, D, C)))))))) ■

Backtrack-free search involves an ordering of the variables.
To achieve adaptive-consistency, variables are processed according to the reverse of this ordering.

For each variable x that is being processed, a k-constraint is created for its Parents, where k is the cardinality of x's parents in the primal graph.

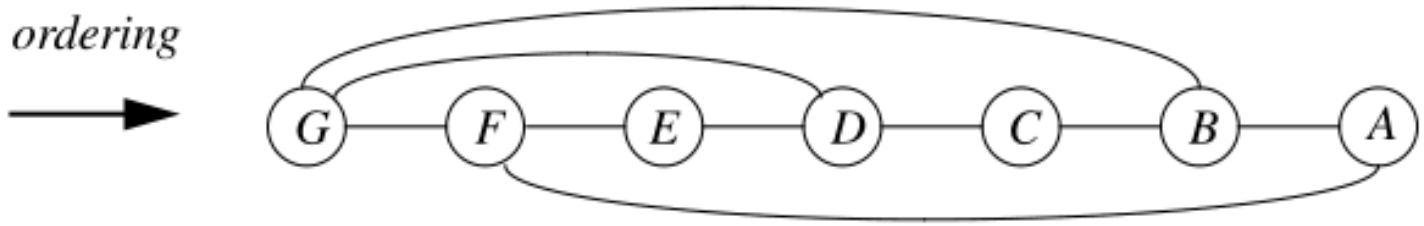
Compound labels in this constraint which are either incompatible with each other or incompatible with all the values in D_x are removed.

Then edges are added between all pairs of nodes in the parents in the primal graph.

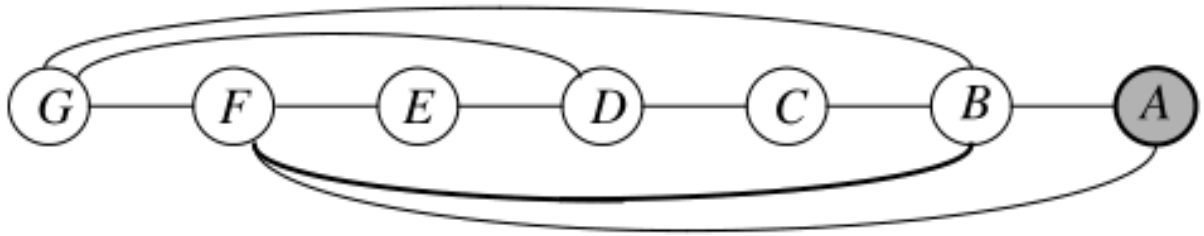
```
PROCEDURE Adaptive_consistency(Z, D, C, <)
/* x_i denotes the i-th variable in Z according to the ordering < */
BEGIN
    FOR i = |Z| to 1 by –1 DO
        BEGIN
            S ← {w | w ∈ Z ∧ w < x_i ∧ (∃ C_X ∈ C: w, x_i ∈ X)};
            C_S ← {cl | cl = compound label for S such that ∃ v_i ∈ D_{x_i}:
                satisfies(cl+<x_i,v_i>, CE(S + {x_i}, (Z, D, C)))};
            C ← C + {C_S};
        END;
    return(Z, D, C, <);
END /* of Adaptive_consistency */
```

The procedure assumes that variables ordering $x_1, x_2, ..., x_n$. These variables are processed in reverse order. When x_i is processed, this removes from the constraint for the parents of x_i all those compound labels which either violate some constraints on the parents or have no compatible values in x_i. Therefore, this deals with j-constraints rather than just binary constraints.

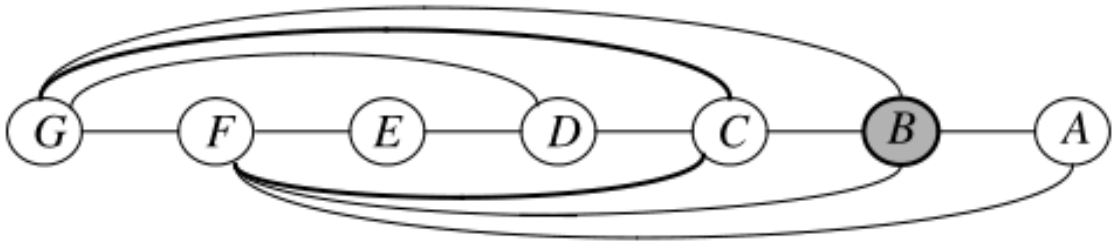
Figure 4.3 Example showing the change of a graph during adaptive-consistency achievement (Processing of E, F and G add no more edges, and therefore the graph shown in (e) is the induced graph. Its width is 3)



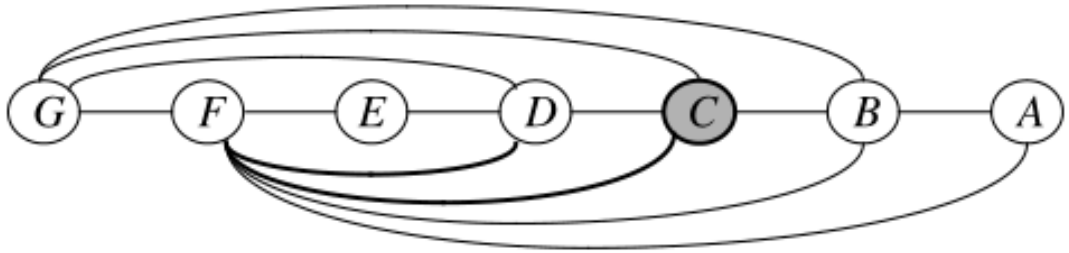
(a) Ordered graph to be processed (from Figure 3.5)
nodes order: (G,F,E,D,C,B,A), process order: (A,B,C,D,E,F,G)



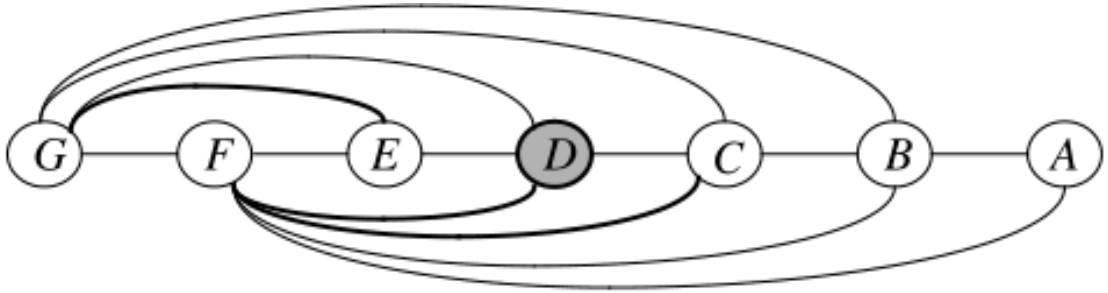
(b) Node A is processed, and edge (F, B) is added



(c) Node *B* is processed, and edges (*G*, *C*) and (F, C) are added



(d) Node *C* is processed, and edge (*F*, *D*) is added



(e) Node *D* is processed, and edge (*G*, *E*) is added

Theorem 4.1
If adaptive-consistency is achieved in a CSP under an ordering, then a search under this ordering is backtrack-free:

$\forall$ csp((Z, D, C)): ($\forall$ <: total_ordering(Z, <):
(adaptive-consistent((Z, D, C),<) $\Rightarrow$ backtrack-free((Z, D, C), <)))

See proof in the textbook.

Definition 4-4:

The primal graph of a CSP P after adaptive-consistency is achieved under some ordering of the variables (i.e. possibly with new edges added) is called the **induced-graph** of P under that ordering, denoted by *induced-graph(P, Ordering)*:

$\forall$ csp((*Z*, *D*, *C*)): ($\forall$ <: total_ordering(*Z*, <):
    ($\forall$ csp((*Z*, *D'*, *C'*)):
        equivalent((*Z*,*D*,*C*), (*Z*,*D'*,*C'*)) $\wedge$ adaptive-consistent((*Z*,*D'*,*C'*),<):
            induced-graph((*Z*, *D*, *C*), <) $\equiv$ G((*Z*,*D'*,*C'*)))) ■

Definition 4-5:

The width of the induced graph of a CSP P under some orderings of its variables is called the **induced-width** of P under that ordering. It is denoted by *induced-width(P, <)*:

$\forall$ csp((*Z*, *D*, *C*)): ($\forall$ <: total_ordering(*Z*, <):
    induced-width((*Z*, *D*, *C*), <) ≡ width(induced-graph((*Z*, *D*, *C*), <)))) ■

Definition 4-6:

The **induced-width** of a CSP P, denoted by *induced-width(P)*, is the minimum induced-width of P under all orderings:

$\forall$ csp((*Z*, *D*, *C*)):
    induced-width((*Z*, *D*, *C*)) ≡
        MIN width(induced-graph((*Z*, *D*, *C*),<))): total_ordering(*Z*, <)) ■

a = the maximum size of the domains in a CSP
W^* = the induced-width of the problem under some ordering <

The time complexity of Adaptive_consistency under < is $O(a^{W^*+1})$
The space complexity is $O(a^{W^*})$.

The optimal ordering which gives W^* (the minimum induced-width of all possible orderings) is NP-hard to compute.

The actual time complexity of the Adaptive_consistency algorithm is hard to compute. Partly because of this, how useful this algorithm is for solving realistic problems is yet to be studied. However, it does give us some insight into the complexity of CSP solving.

4.7 Parallel/Distributed Consistency Achievement

4.7.1 A connectionist approach to AC achievement

A *connectionist approach* to problem solving is to represent the problem with a network, where each node is implemented by a piece of hardware which is only required to perform very simple tasks.

Efficiency is gained by making use of a large number of (simple) processors and the carefully chosen connections. Connectionist approaches to CSP solving will be revisited later.

AC-3 and AC-4 are based on the notion of support.
A label <x,*a*> is supported if for every variable *y* there exists a value *b* such that <y,*b*> is legal and (<x,*a*><y,*b*>) satisfies the constraint $C_{x,y}$.

Given a binary CSP, a network is set up in the following way:

- a *v-node* is used to represent each variable,
- a *c-node* is used to represent a 2-compound label, regardless of whether the 2-compound label satisfies the relevant constraints.

Each node in the network may take a binary value (0 or 1), indicating whether this label or compound label is legal. For variables x, x_1 and x_2 we use

- $v(<x,a>)$ to denote the value taken by the v-node which represents the label $<x,a>$,
- $c(<x_1,v_1>\ <x_1,v_2>)$ to denote the value taken by the c-node which represents the 2-compound label ($<x_1,v_1>\ <x_2,v_2>$).

Each pair of v-nodes which represent the labels $<x_1,v_1>$ and $<x_2,v_2>$ such that $x_1 \neq x_2$ are connected through an AND gate to the c-node which represents the 2-compound label ($<x_1,v_1><x_2,v_2>$). In other words, c($<x_1,v_1><x_2,v_2>$) will be set to 0 if either of $<x_1,v_1>$ or $<x_2,v_2>$ is 0.

Each v-node $<x_1,v_1>$ is connected by all the c-nodes which represent 2-compound labels ($<x_1,v_1>$ $<x_2,v_2>$) for some x_2 ($\neq x_1$) and v_2 under the following logic:

$$v(<x_1,v_1>) \leftarrow v(<x_1,v_1>) \wedge \bigcap_{x_2 \in Z \wedge x_1 \neq x_2} \left(\bigcup_{v_2 \in D_{x_2}} v(\langle x_2, v_2 \rangle) \wedge c(\langle x_1, v_1 \rangle \langle x_2, v_2 \rangle) \right)$$

where Z is the set of variables in the problem and D_{x2} is the domain of x_2.

The network is initialized in such a way that all the v-nodes are set to 1 and all the c-nodes are set to 1 if the compound label that it represents satisfies the constraint on the variables; it is set to 0 otherwise.

After initialization, the network is allowed to converge to a stable stage. A network will always converge because nodes can only be switched from 1 to 0, and there are only a finite number of nodes.

When the network converges, all the v-nodes which are set to 0 represent labels that are incompatible with all the values of at least one other variable (because of the set up of the network).

The soundness and completeness of this network follow trivially from the fact that its logic is built directly from the definition of AC.

The space complexity of this approach is $O(n^2a^2)$, where n is the number of variables and a is the largest domain size in the problem.

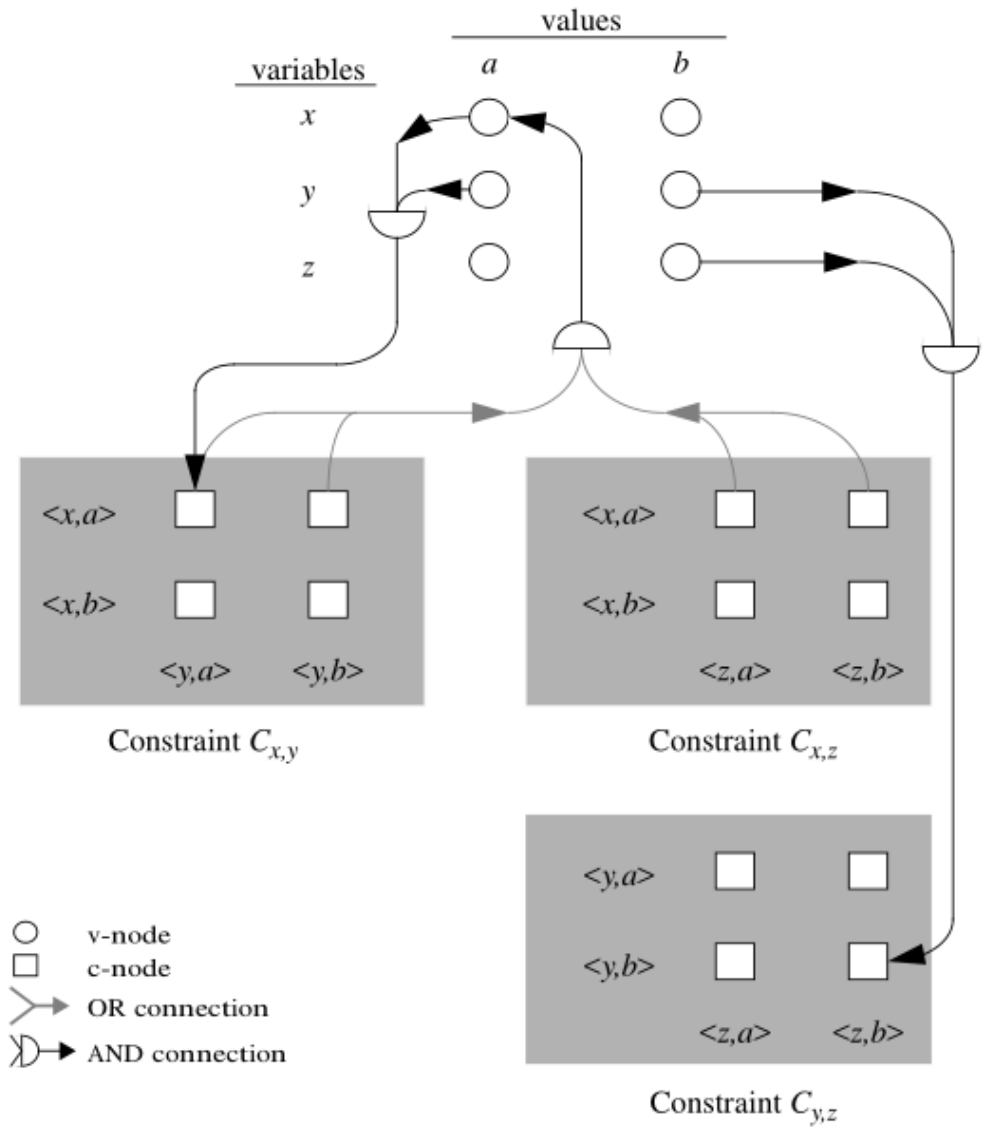


Figure 4.4 A connectionist representation of a binary CSP (Z, D, C), where the variables $Z = \{x, y, z\}$ and all the domains are $\{a, b\}$ (only three sets of connections are shown here)

4.7.2 Extended parallel arc-consistency

After convergence, AC is achieved in the network described in the previous section. Guesgen & Hertzberg [Gues91] [GueHer92] propose a method that stores information in the network which can help in solving the CSP.

Each c-node is given a *signature*, which could be a unique prime number.
Each v-node is made to store a set of signatures.
Each c-node stores a binary value as before, but it outputs a set of signatures, as explained later.
For convenience, we call the signatures of the c-node for the compound label (<x_1,v_1> <x_2,v_2>) *s*(<x_1,v_1> <x_2,v_2>).
The connections remain the same as before.
Each v-node representing <*x*,*v*> is initialized to the set of all signatures for all the c-nodes except those which represent 2-compound labels involving <*x*,*v*'> with $v' \neq$ v.

The initial values for the c-nodes are the same as the network described earlier, i.e. a c-node is set to 1 if the compound label that it represents is legal, and 0 otherwise.

Guesgen calls the convergence mechanism *graceful degradation*. Each v-node outputs the signatures that it stores to its connected c-nodes.

When a c-node is on, it takes the intersection of the (two) input sets of signatures and outputs the result to the v-nodes connected to it; an empty set is output if it is off.

Therefore, input to each v-node is sets of signatures from the c-nodes. These inputs will be combined using the logic shown in Figure 4.4, with AND operations replaced by set union operations, and OR operations replaced by set intersection operations.

This combined result is intersected with value currently stored in v-node.

For the c-node for (<x, a> <y, b>):

input:	$v(\langle x, a\rangle) \cap v(\langle y, b\rangle)$	
value (static):	1	if (<x, a> <y, b>) satisfies $C_{x,y}$;
	0	otherwise
output:	$v(\langle x, a\rangle) \cap v(\langle y, b\rangle)$	if c(<x, a> <y, b>) = 1;
	{ }	otherwise

For the v-node for <x, a>:

input:	$\bigcap_{y \in Z} \bigcup_{b \in D_y}$ output of the c-node for (<x, a> <y, b>)
value:	current value $\cap$ input
output:	current value (i.e. v(<x, a>))

Figure 4.5 Summary of the input, output and values of the nodes in Guesgen's network (Z = set of variables, D_y = domain of y)

The network will always converge because the number of signatures stored in the v-nodes is finite and non-increasing.
After the network has converged, a solution W to the CSP is a compound label such that:

1. W contains exactly one label per variable; and
2. for every label w in W, v(w) contains the signatures of all the c-nodes which represent some 2-compound labels which are projections of W. In other words, $\forall\, w \in W$: $P \subseteq \text{v}(w)$, where $P = \{\text{s}(l_1,l_2) \mid l_1 \in W \wedge l_2 \in W \wedge l_1 \neq l_2\}$

Guesgen & Hertzberg. Do not suggest how this compound label W can be found. If an algorithm does find Ws which satisfy the above conditions, then this algorithm is sound and complete for binary CSPs.

Let n be the number of variables and a be the maximum size of the domains. There are na v-nodes, and n^2a^2 signatures.
The space complexity of Guesgen's algorithm for the network is n^3a^3.

4.7.3 Intractability of parallel consistency

Kasif [1990] points out that the problem of achieving AC and the problem of testing the satisfiability of propositional Horn clauses belong to the same class of problems which are logarithmic-space complete.

AC-consistency is unlikely to be achievable in less than logarithmic time by massive parallelism.

CSPs cannot be solved in logarithmic time by using only a polynomial number of processors.

Linear speed up is likely to be achievable when the number of processors is significantly smaller than the size of the constraint graph (which is often true).

4.8 Summary

n = number of variables; e = number of binary constraints; a = size of the largest domain		
Algorithm	Time complexity	Space complexity
NC-1	$O(an)$	$O(an)$
AC-1	worst case: $O(a^3ne)$	$O(e+na)$
AC-3	lower bound: $\Omega(a^2e)$ upper bound: $O(a^3e)$	$O(e+na)$
AC-4	worst case: $O(a^2e)$	$O(a^2e)$
DAC-1	worst case: $O(a^2e)$; or $O(a^2n)$ when the constraint graph forms a tree	$O(e+na)$

PC-1	worst case: $O(a^5n^5)$	$O(n^3a^2)$
PC-2	lower bound: $\Omega(a^3n^3)$, upper bound: $O(a^5n^3)$	$O(n^3+n^2a^2)$
PC-4	worst case: $O(a^3n^3)$	$O(n^3a^3)$
DPC-1	worst case: $O(a^3n^3)$	$O(n^2a^2)$
KS-1 (to achieve k-consistency)	worst case : $O\left(\sum_{i=1}^{k} ({}_nC_i \cdot a^i)\right)$	$O(\sum_{i=1}^{k} ({}_nC_i \cdot a^i))$
Adaptive_ consistency	worst case: $O(a^{W^*+1})$, where W^* = induced-width of the constraint graph (W^* is NP-hard to compute)	$O(a^{W^*})$, where W^* = induced-width